CS612 · ADVANCED ALGORITHMS · MSC · FALL 2026

Lecture 10
Shortest Paths as an Algorithm-Selection Problem

Don't memorise five algorithms. Learn which assumption each one needs, and what breaks without it.

Saturday 28 November 2026 · 3 hours

Project M4 (working implementation) due today · PS3 due Sat 05 Dec

Motivating problem: one product, five problems

A logistics platform needs "shortest paths" in five places:

	Setting	What is special
1	Hops between warehouses in a courier network	unweighted
2	Earliest completion in a task dependency plan	a DAG; the *longest* path is wanted
3	Travel time on the road network	non-negative weights, sparse, huge
4	Net cost with fuel rebates	some *negative* edges
5	Currency conversions for payouts	a negative *cycle* = arbitrage

DISCUSS · 3 MIN

For which of these is Dijkstra's algorithm **wrong**, and for which is it merely **slower than necessary**?

BLOCK A · 00:10–01:00

Choosing by assumption

Relaxation · BFS · DAGs · Dijkstra · small integer weights

The assumption map

Assumption	Single source	All pairs
Unweighted	BFS $O(n+m)$	n × BFS: $O(n(n+m))$
DAG (any weights)	topological relaxation $O(n+m)$	n × that
Integer weights in $[0, C]$	Dial buckets $O(m+nC)$; 0/1 weights: 0-1 BFS $O(n+m)$	n × that
Non-negative weights	Dijkstra: $O((n+m)\log n)$ binary heap; $O(n^2)$ array; $O(m+n\log n)$ Fibonacci heap	n × Dijkstra, or Floyd–Warshall if dense
Negative edges, no negative cycle	Bellman–Ford $O(nm)$	Johnson $O(nm+n(n+m)\log n)$; Floyd–Warshall $O(n^3)$
Negative cycles possible	detect with Bellman–Ford (or FW: $d[i][i]<0$); the *shortest simple path* is NP-hard	

> Read the table **top-down**: use the first row whose assumption your instance satisfies. Every row down pays for a weaker assumption.

Relaxation: the one operation they all share

```
RELAX(u, v)
  if d[u] + w(u,v) < d[v]
     d[v] = d[u] + w(u,v) ; parent[v] = u
```

Initialise $d[s] = 0$ and $d[v] = \infty$ otherwise. Let $\delta(v)$ be the true distance.

UPPER-BOUND INVARIANT

At all times $d[v] \geq \delta(v)$, and $d[v]$ (when finite) is the length of some actual s–v walk. It never increases.

PATH-RELAXATION PROPERTY

If $s = v_0, v_1, \ldots, v_k$ is a shortest path and its edges are relaxed **in this order** (other relaxations may be interleaved), then afterwards $d[v_k] = \delta(v_k)$.

EACH ALGORITHM IS A RELAXATION ORDER

- **DAG:** topological order: every path's edges are relaxed in order.
- **Dijkstra:** in order of final distance, which needs $w \geq 0$.
- **Bellman–Ford:** all edges, $n - 1$ times, so any path of $\leq n - 1$ edges gets its edges relaxed in order.

Dijkstra: why non-negative weights are essential

Dijkstra repeatedly extracts the unvisited vertex u with minimum $d[u]$, marks it final, and relaxes its out-edges.

CLAIM

When u is extracted, $d[u] = \delta(u)$.

PROOF

Suppose not, and let u be the first extracted vertex with $d[u] > \delta(u)$. Take a shortest s–u path and let y be its first vertex not yet extracted, with predecessor x (extracted, so $d[x] = \delta(x)$). Since x was relaxed, $d[y] = \delta(y)$. The rest of the path from y to u has **non-negative length**, so $\delta(y) \le \delta(u)$. Hence $d[y] = \delta(y) \le \delta(u) < d[u]$, yet u was extracted before y (the minimum): a contradiction. ∎

Priority queue	insert / decrease	extract-min	Dijkstra total
array	$O(1)$	$O(n)$	$O(n^2 + m)$
binary heap (lazy or decrease-key)	$O(\log n)$	$O(\log n)$	$O((n+m)\log n)$
Fibonacci heap	$O(1)$ amortized	$O(\log n)$ amortized	$O(m + n\log n)$

Cheaper special cases: DAGs and small integer weights

DAG shortest (or longest) paths

```
for u in topological order
    for each edge (u, v): RELAX(u, v)
```

Correct for **any** weights: by the path-relaxation property, every path's edges are relaxed in order. $O(n + m)$. The longest path in a DAG = the shortest with negated weights: **critical path** scheduling.

Integer weights in $[0, C]$

- **Dial:** buckets indexed by distance (a circular array of $C + 1$ buckets). $O(m + nC)$ (pseudo-polynomial in C: L07!).
- **0-1 BFS:** weights in $\{0, 1\}$. Use a deque: push the target of a 0-edge to the front and of a 1-edge to the back. $O(n + m)$.
- Radix heaps: $O(m + n \log C)$, polynomial in the input length.

> Longest paths are easy in DAGs and **NP-hard** in general graphs. The DAG assumption is doing all the work.

Exercise A: select and justify

1. **Arbitrage.** Exchange rates r_{ij} between 150 currencies. Detect whether a cycle of conversions gains money. Model it as a shortest-path problem and choose the algorithm.
2. **Grid game.** A 2000×2000 grid; entering a cell costs 0 (road) or 1 (mud). Minimum cost from corner to corner.
3. **Build plan.** 50,000 build tasks with durations and dependencies. What is the earliest possible completion time of the whole build?
4. **Continental routing.** $n = 2 \times 10^7$ road junctions, non-negative travel times, millions of queries per hour. Which algorithm per query? What would you add beyond this lecture?

Pairs · 8 min

BLOCK B · 01:10–02:00

Negative weights and all pairs

Bellman–Ford · potentials · Johnson · Floyd–Warshall

Bellman–Ford: correctness and negative cycles

```
BELLMAN-FORD(G, s)
  initialise d
  repeat n − 1 times
    for each edge (u,v): RELAX(u,v)
    if nothing changed: stop early
  for each edge (u,v)
    if d[u] + w(u,v) < d[v]: report "negative cycle reachable from s"
```

Correctness: with no negative cycle reachable from s, every vertex has a shortest path that is *simple*, with $\leq n-1$ edges. Round i relaxes its i-th edge in order (path relaxation). **Detection:** if no edge can be relaxed, summing $d[v] \leq d[u] + w(u,v)$ around any reachable cycle gives cycle weight ≥ 0.

PRACTICAL POINTS

- **Early termination:** the number of rounds is at most (edges on a shortest-path tree's deepest path) + 1. In our experiment: 15 rounds, not 200,000.
- **Whole-graph detection:** add a virtual source with 0-edges to every vertex.

LEGACY CORRECTIONS

- "Run BF per connected component" does not work for directed graphs: use the virtual source.
- The queue-based variant stopped after n *total* pops and reported a cycle. That is wrong: count relaxations *per vertex* (or path lengths ≥ n).
- An undirected edge of negative weight is itself a negative cycle.

Potentials and reduced costs

REDUCED COST

For any function $h : V \to \mathbb{R}$ (a *potential*), let $w_h(u, v) = w(u, v) + h(u) - h(v)$.

Telescoping: for any s–t path P: $w_h(P) = w(P) + h(s) - h(t)$. Every s–t path shifts by the **same** constant, so shortest paths are preserved. Cycles keep their weight.

Feasible potential: $w_h \geq 0$ everywhere. It exists **iff** there is no negative cycle.

How to find one: $h(v) = \delta(q, v)$ from a virtual source q. The triangle inequality $\delta(q, v) \leq \delta(q, u) + w(u, v)$ gives $w_h(u, v) \geq 0$.

WHY THIS MATTERS

- Johnson's algorithm (next slide)
- **A*:** a heuristic π is *consistent* iff $w_{-\pi} \geq 0$ (reduced costs non-negative); A* = Dijkstra on reduced costs
- Min-cost flow: potentials keep residual costs non-negative (L12 outlook)
- LP duality: potentials are dual variables

Johnson's algorithm and Floyd–Warshall

Johnson (1977)

1. Add a virtual source q; run Bellman–Ford: $h(v) = \delta(q, v)$, or report a negative cycle. $O(nm)$.
2. Reweight to $w_h \geq 0$.
3. Dijkstra from every vertex on w_h: $n \cdot O((n+m)\log n)$.
4. $\delta(u, v) = \delta_h(u, v) - h(u) + h(v)$.

Total $O(nm + n(n+m)\log n)$ with a binary heap; $O(nm + n^2 \log n)$ with a Fibonacci heap. Best for **sparse** graphs.

Floyd–Warshall

$D_k[i][j]$ = the shortest i–j path with intermediate vertices in $\{1..k\}$:

$$D_k[i][j] = \min\left(D_{k-1}[i][j],\ D_{k-1}[i][k] + D_{k-1}[k][j]\right).$$

In-place update is safe (row and column k do not change in round k without negative cycles). $\Theta(n^3)$ time, $\Theta(n^2)$ space; simple and cache-friendly. Negative cycle $\Leftrightarrow$ some $D[i][i] < 0$ at the end.

> Sparse ($m = O(n)$): Johnson $\approx n^2 \log n \ll$ FW n^3. Dense ($m \approx n^2$): both $\approx n^3$, and FW's simplicity usually wins.

Exercise B: reasoning with assumptions

1. Bellman–Ford with early termination on a graph whose shortest-path tree has depth d: how many rounds? Can the edge order within a round make it faster? (Try an order for a path graph.)
2. Johnson on a graph that has **no** negative edges: is step 1 needed? What potential does it compute?
3. Show that Floyd–Warshall computes the **transitive closure** if you replace $(\min, +)$ by $(\vee, \wedge)$. What other "semiring" problems fit the same loops?
4. Shortest path using **at most k edges** (e.g. a flight search limited to k legs). Modify Bellman–Ford. Which subtle bug must you avoid inside a round?

Pairs · 8 min

BLOCK C · 02:10–02:55

Evidence and design

Three measurements · one design exercise

Experiments: assumptions and implementations, measured

1. Sparse, $n = 2 \times 10^5$, $m = 10^6$, integer weights in $[0, 100]$ (ms): BFS 30.6 (wrong answer) · Dijkstra, binary heap 107.2 · Dial 31.0 · Bellman–Ford 109.5 (**15 rounds**, not n). All weighted algorithms agree.

2. Dijkstra, $n = 5000$ (ms)

m/n	binary heap	array $O(n^2)$
5	0.7	22.6
50	3.8	30.4
500	10.1	31.5
4999	72.8	131.4

3. All pairs, negative edges, $n = 1200$ (ms)

m/n	Johnson	Floyd–Warshall
4	168	1229
40	485	1673
400	1399	1705

Random digraphs; negative weights created by a random potential (so no negative cycles); `Student_Resources/sp_bench.cpp`, g++ 15.2 -O2.

Exercise C: design with assumptions

1. **Time-dependent tolls**: an edge's cost is $w(u, v)$, plus a toll $t_v \geq 0$ charged at a vertex only on weekdays; queries say whether it is a weekday. Two options: preprocess, or adapt at query time. Which do you choose?
2. **Unit-capacity reliability**: each edge fails independently with probability p_e. Find the path most likely to survive. Reduce it to a shortest-path problem, and name the algorithm and the assumption you rely on.
3. **Widest path.** Each edge has a capacity. Find the s–t path that maximises the minimum capacity along it. Modify Dijkstra, and identify the property of $\min$ that replaces "non-negative weights" in the proof.
4. **An experiment you would run** to decide between Dial and a binary heap for road networks with integer travel times in seconds ($C \approx 3600$). What is your hypothesis, and what would refute it?

Groups · 15 min

Synthesis

1. Shortest paths are one problem with many assumptions: unweighted, DAG, small integers, non-negative, negative edges, negative cycles, sparse/dense, single/all pairs.
2. All algorithms are relaxation orders. The proofs use the upper-bound invariant + path relaxation, and Dijkstra adds one line that needs $w \geq 0$.
3. Name the priority queue: array n^2, binary heap $(n + m) \log n$, Fibonacci $m + n \log n$.
4. Potentials make negative edges harmless when there are no negative cycles (Johnson), and they explain A*.
5. Measurements can contradict worst-case intuition (heap Dijkstra on dense random graphs). Know which input families you tested.

NEXT WEEK · L11 (SAT 05 DEC)

String algorithms: rolling hashes and collision probability, Rabin–Karp, KMP, tries, Aho–Corasick.

Read: CLRS 4e Ch. 32. **PS3 due Sat 05 Dec.**